

Technical debt

Technical debt is the implied cost of future rework in software development from choosing easy, quick solutions now instead of better, more robust approaches, akin to taking a financial loan for immediate gain but incurring interest later.

It's caused by shortcuts, poor documentation, or outdated code, leading to slower development, increased bugs, and higher maintenance costs over time, requiring "repayment" through refactoring or debugging, which can be intentional for quick releases or accidental.

Key Aspects of Technical Debt

- **The Analogy:** Like financial debt, you get immediate benefits (faster release), but must "pay interest" (extra work, slower progress) later to fix or improve the code.
- **Causes:** Rushing to meet deadlines, poor design choices, lack of testing, or simply evolving business needs that outpace the original code.
- **Types:** Can be strategic (intentional, like a proof-of-concept) or unintentional (accidental, due to oversight or poor practices).
- **Consequences:** Decreased code quality, reduced maintainability, increased bugs, development slowdown, and potential system instability.

Managing Technical Debt

- **Repayment:** Involves refactoring (restructuring code), debugging, and adding documentation.
- **Balancing Act:** Teams must balance short-term feature delivery with long-term code health.
- **Strategies:** Proactively allocate time to address debt (e.g., 20% of development time) or address it when it becomes a major blocker.